

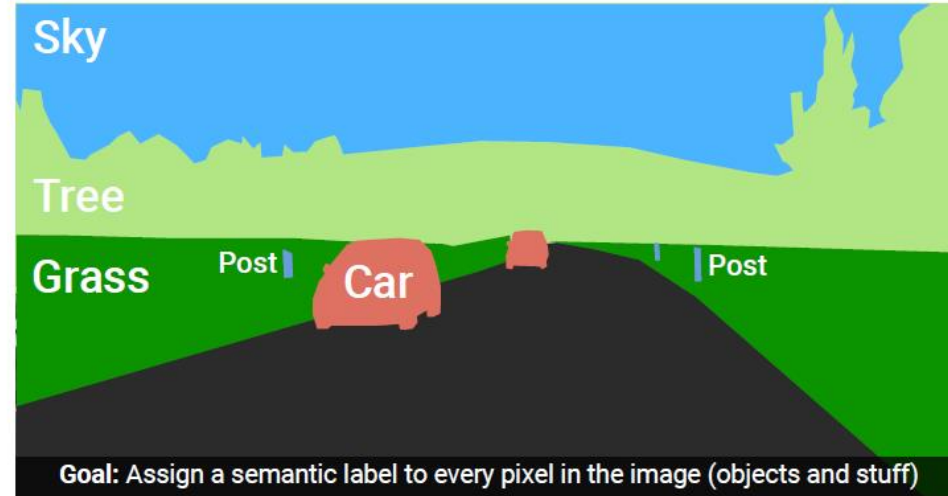
Segmentation and Detection

Computer Vision

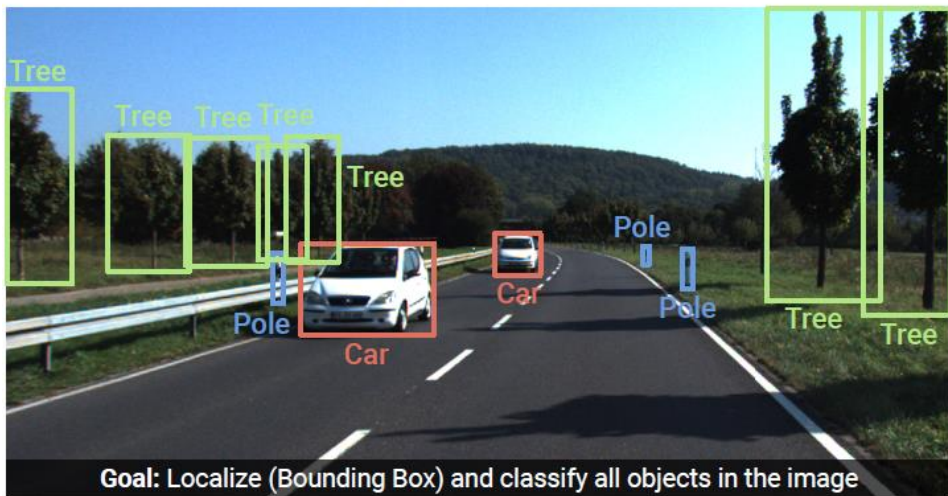
Image Understanding (Recognition)



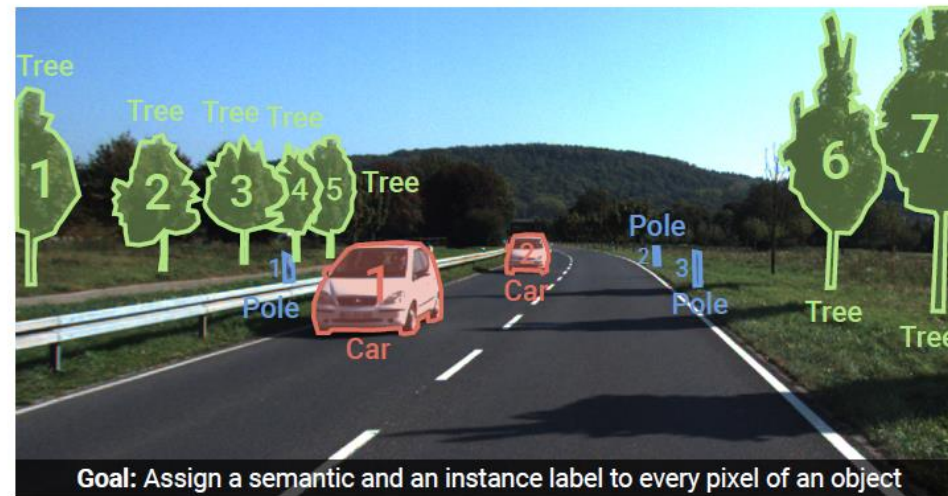
Image Classification



Semantic Segmentation



Object Detection



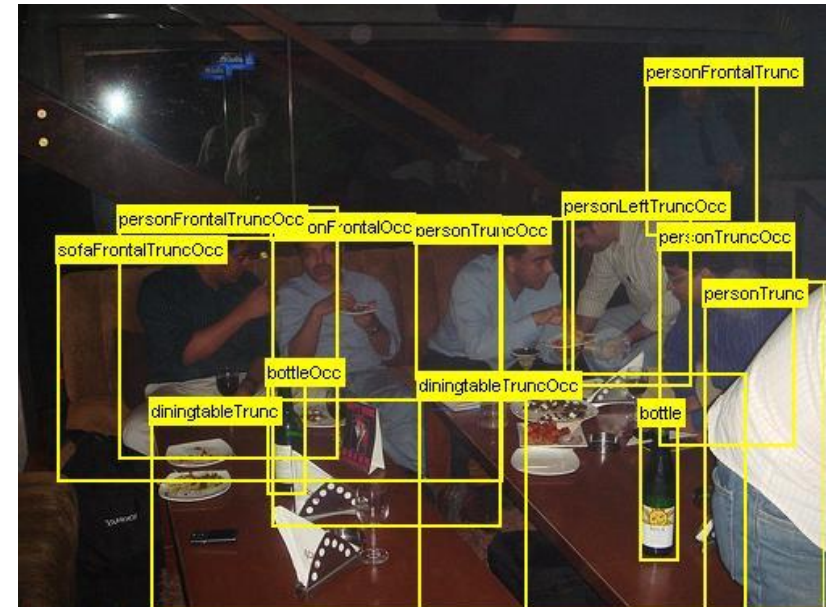
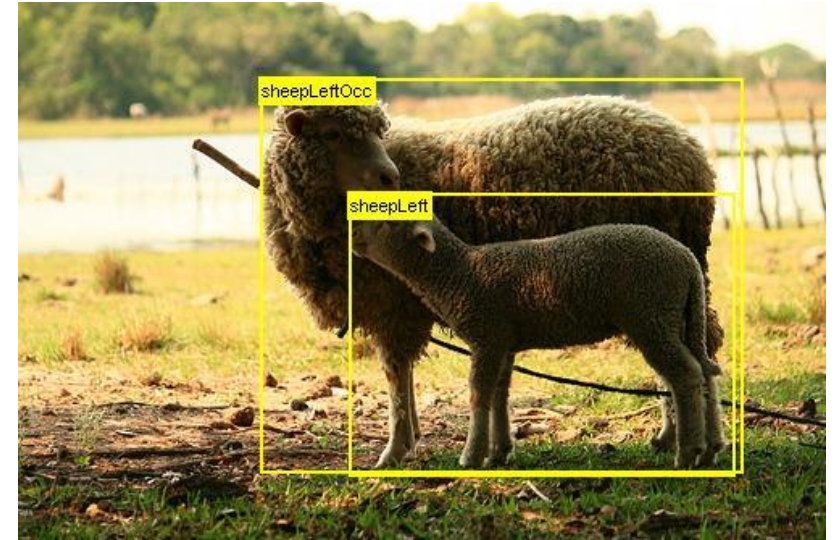
Instance Segmentation

- combination of both: panoptic segmentation

A Few More Image Data Sets

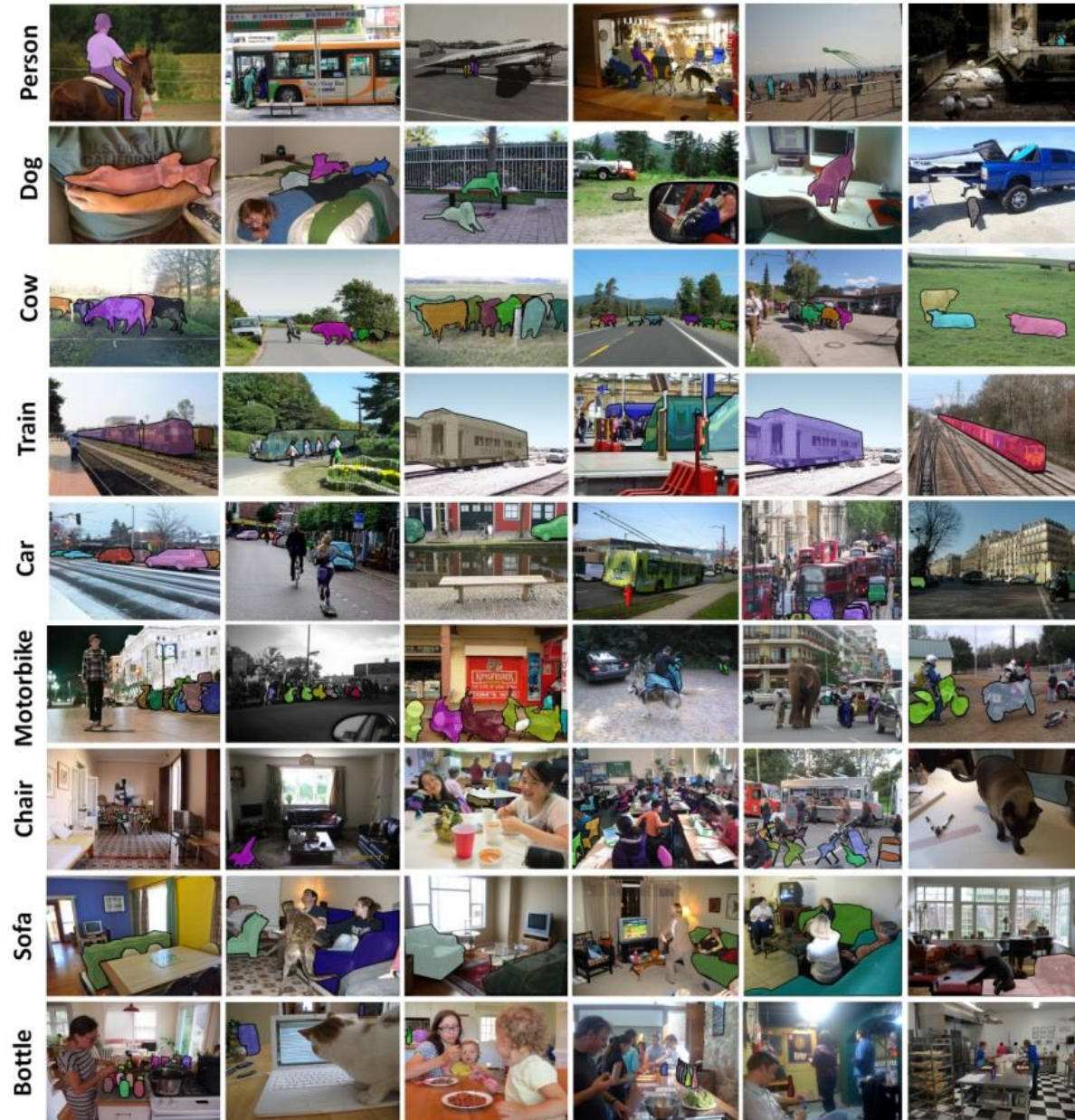
PASCAL VOC Data Set

- PASCAL Visual Object Class challenge
- widely used as benchmark for object detection and semantic segmentation
- 20 object categories such as person, sofa, sheep, car, ...
- 11530 annotated images
- available annotations: pixel-level segmentation, bounding boxes, object classes



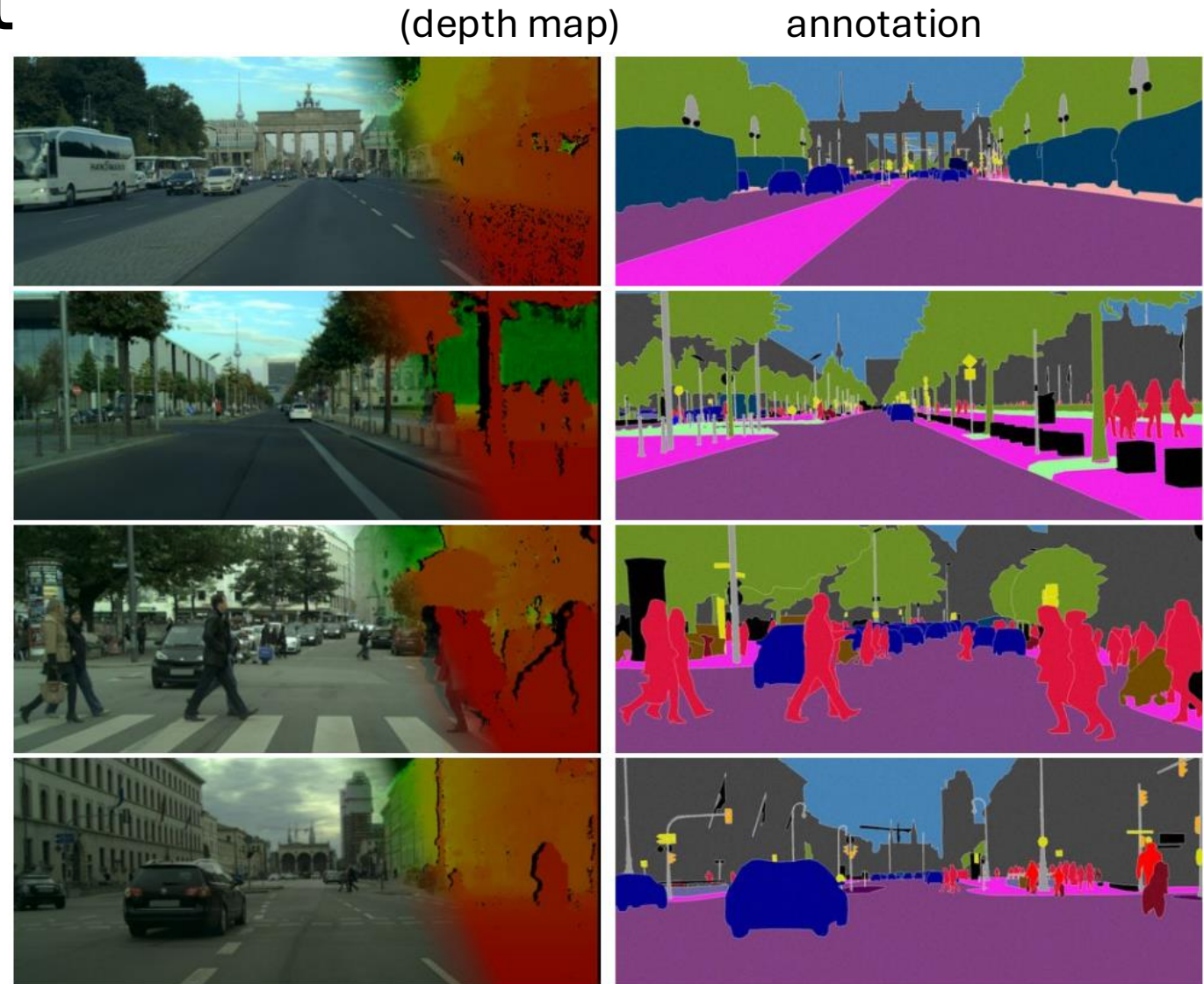
MS COCO Data Set

- Microsoft Common Objects in Context
- images of complex everyday scenes containing common objects in their natural context
- 91 objects types
- 2.5 million annotated instances in 328k images → instance segmentation



Cityscapes Data Set

- goal: semantic understanding of urban street scenes (captured in 50 cities)
- pixel annotations for 30 classes (person, car, building, ...)
- 5000 fine-annotated and 20000 coarse-annotated images



Semantic Segmentation

Object Segmentation from DINO

thresholding self-attention map of last layer:

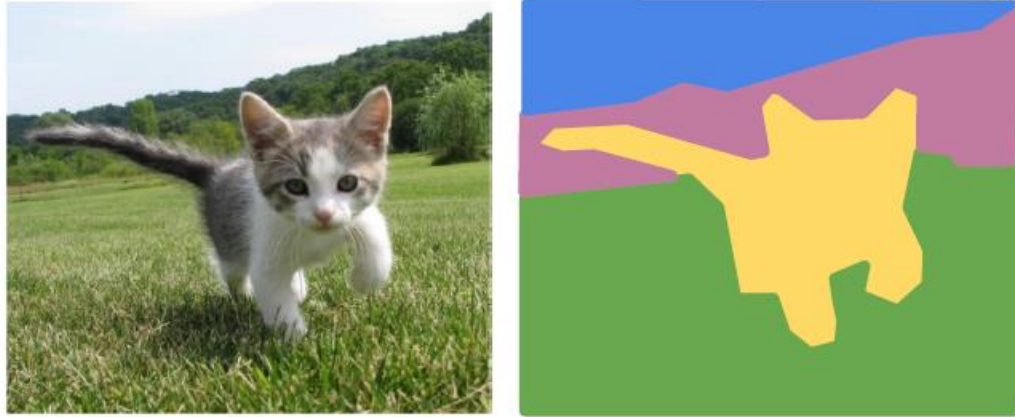


[source](#)

not a full segmentation mask though ...

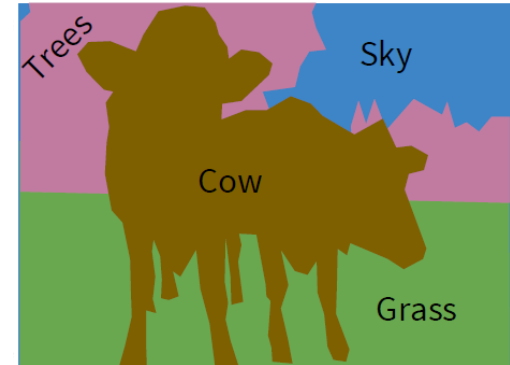
Classification of Each Pixel

segmentation: no objects, just pixels



GRASS, CAT, TREE,
SKY, ...

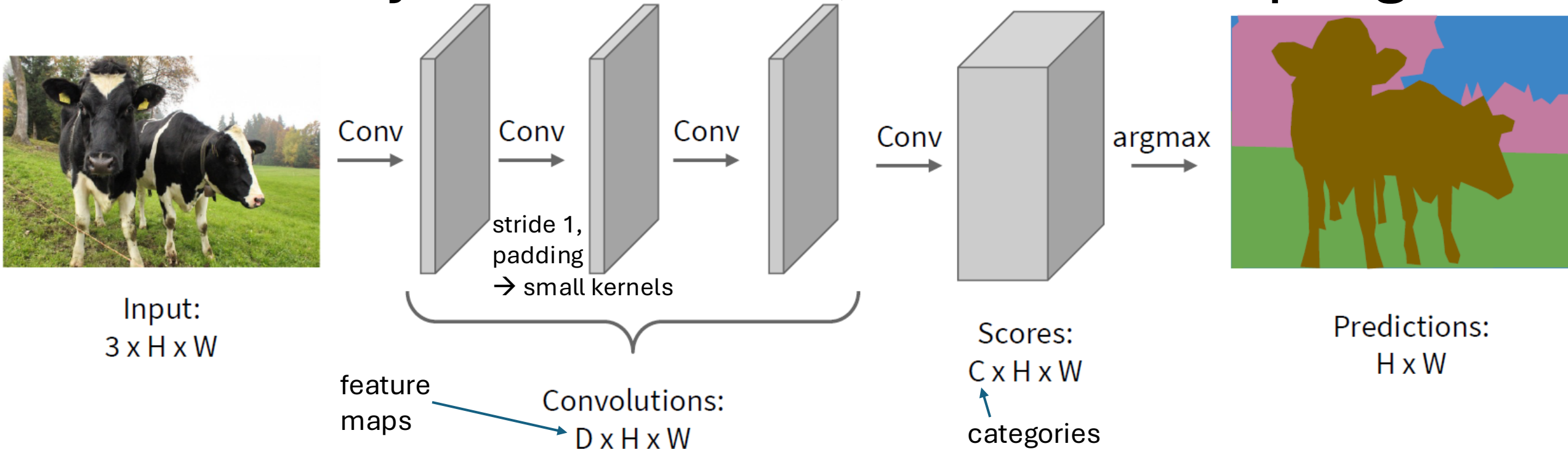
Paired training data: for each training image, each pixel is labeled with a semantic category.



At test time, classify each pixel of a new image.

minimize sum over classification losses
(cross-entropy loss at every output pixel)

Idea: Fully Convolutional, No Downsampling



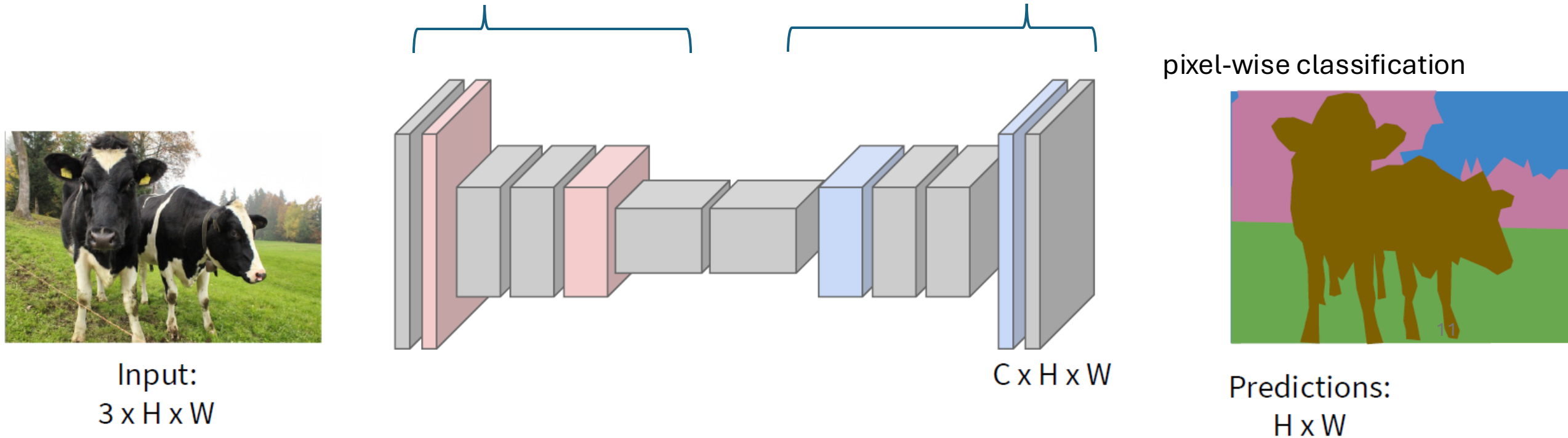
replace flattened, fully-connected classification layers with 1×1 convolutions
 $\rightarrow$ maintain spatial relationships and enable pixel-wise classification:
conversion of feature maps into classification heat maps (one for each class)

but no downsampling means small receptive field and no hierarchical learning

Upsampling to the Rescue

typical spatial feature extraction: convolution and pooling layers $\rightarrow$ downsampling

upsampling layers to restore original image size



different options for down- (pooling, strided convolution) and upsampling ...

Reverse Pooling

resampling (no learned parameters)

Nearest Neighbor

1	2
3	4

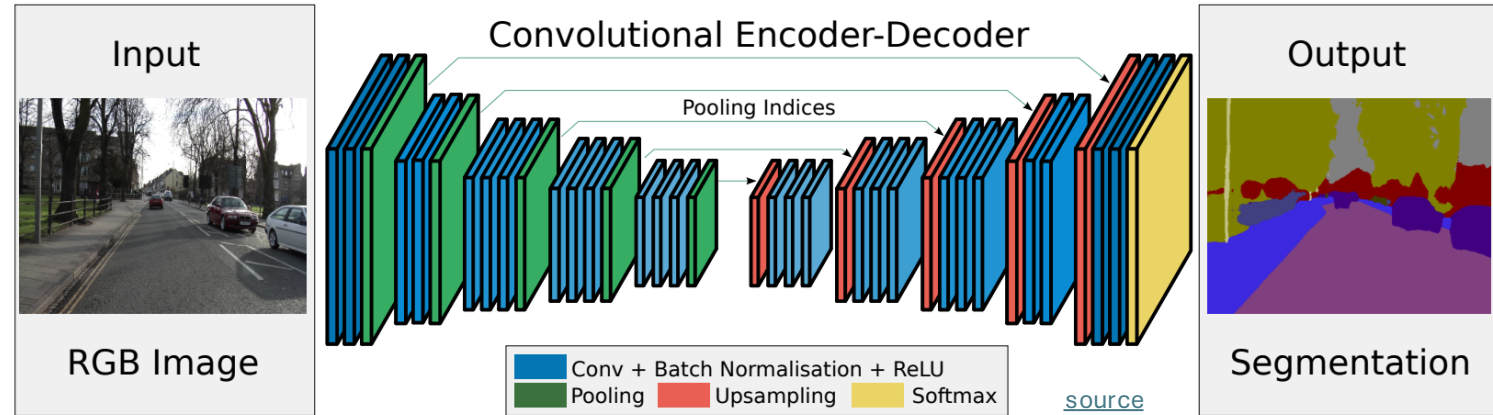


1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

unpooling (recording max positions from pooling)



Max Pooling
Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4

5	6
7	8

Output: 2 x 2

Rest of the network

Max Unpooling
Use positions from
pooling layer

1	2
3	4

Input: 2 x 2

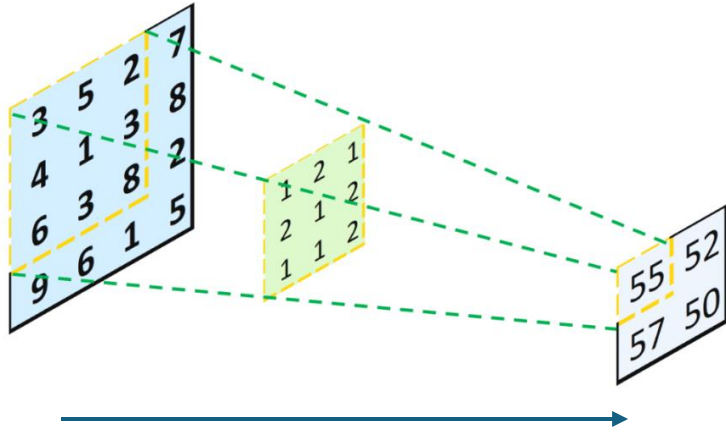
0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Output: 4 x 4

filled with learned parameters in subsequent convolution

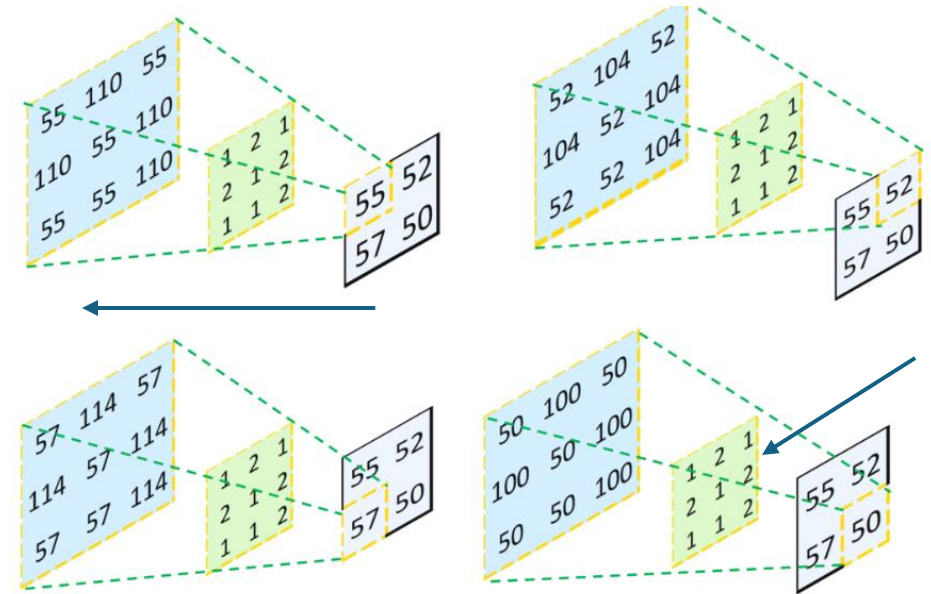
Reverse Convolution

convolution

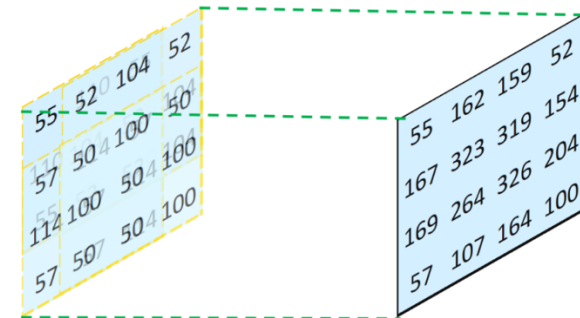


not inverse convolution
→ additional learned parameters

transposed convolution

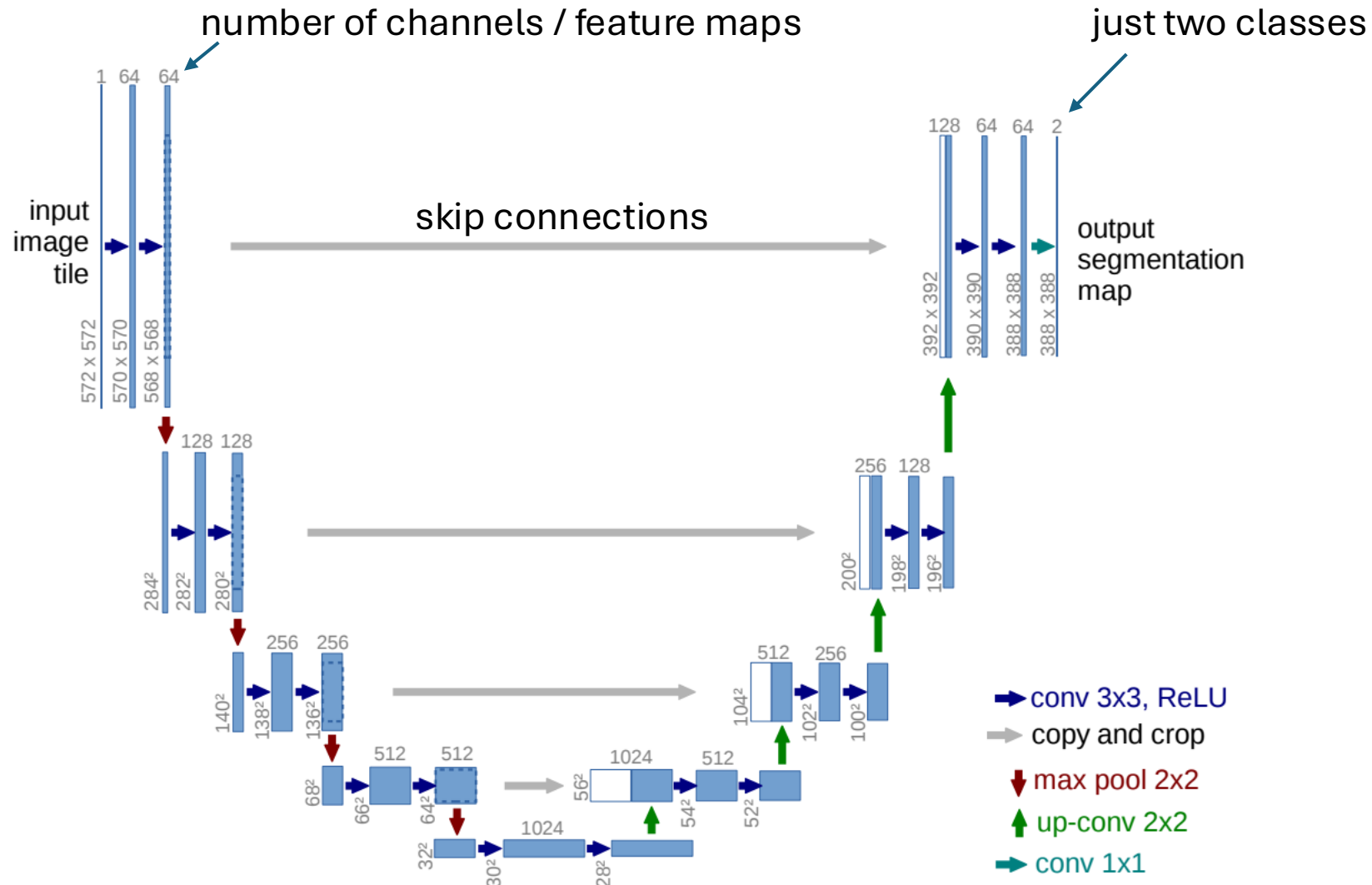


combine and sum overlaps:



[source](#)

U-Net



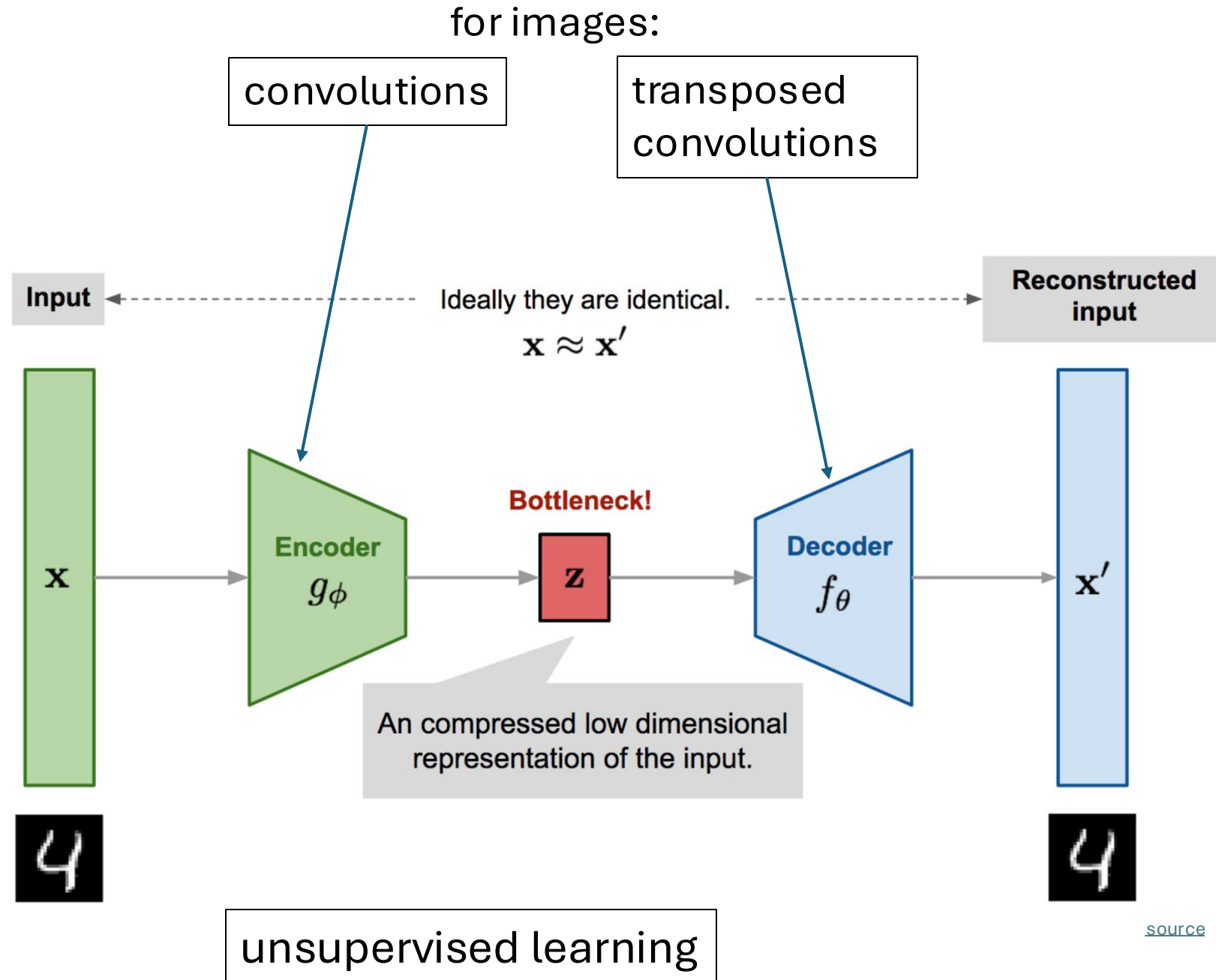
also used for learning of
depth maps, image
synthesis (diffusion), ...

Aside: Autoencoders

Autoencoder

(deep) encoder network
(deep) decoder network
learned together by
minimizing differences
between original input
and reconstructed input
(expressed as losses)

compressed intermediate
representation:
dimensionality reduction
(alternative to PCA)

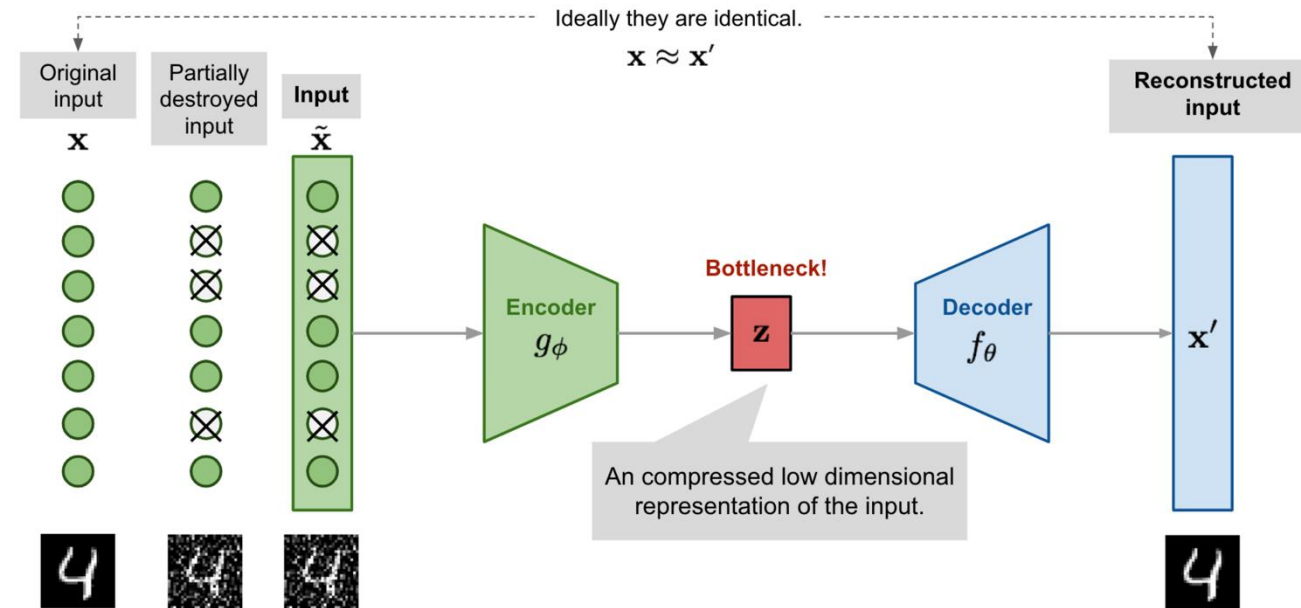


Denoising Autoencoder

goal: avoid overfitting and improve robustness of plain autoencoder

learn to remove noise of distorted input $\tilde{x} \rightarrow$ restore original input x

similar to dropout

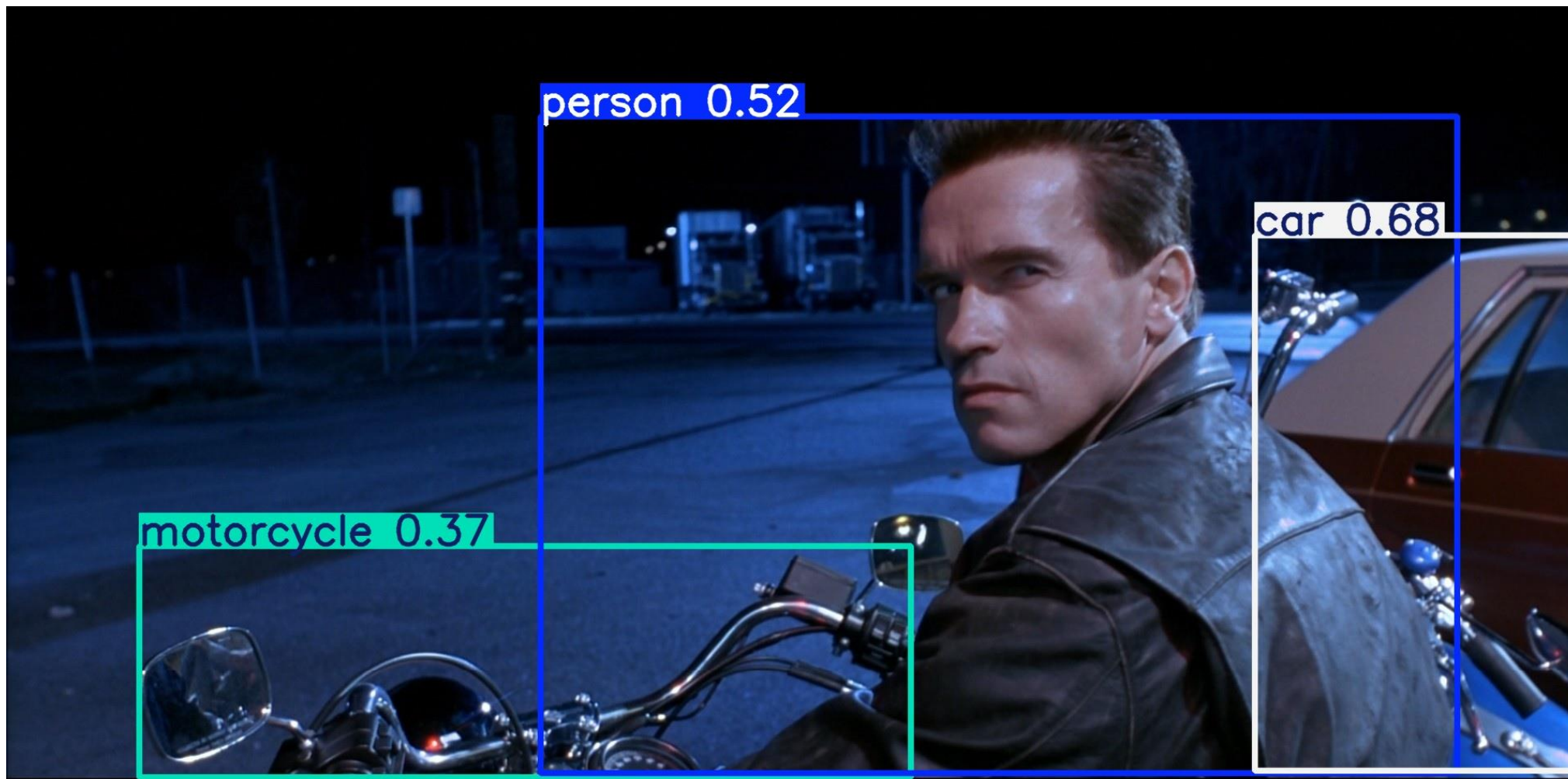


[source](#)

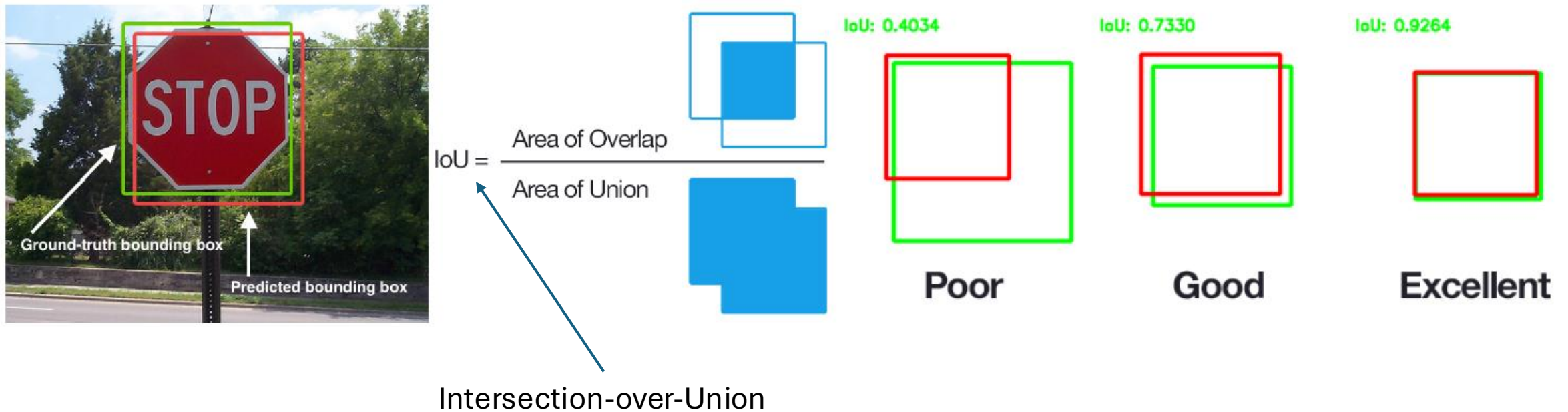
alternative to deconvolution (image restoration)

Object Detection

output: bounding boxes with category label and confidence



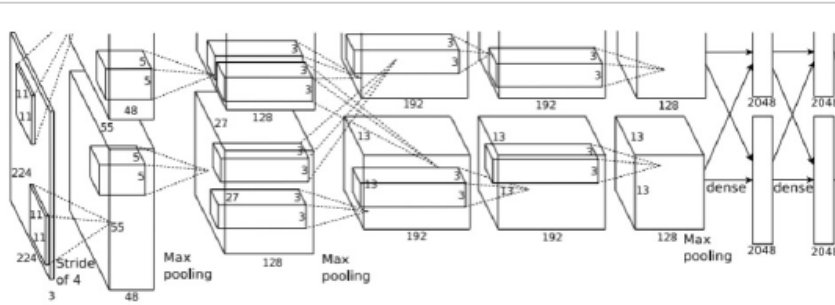
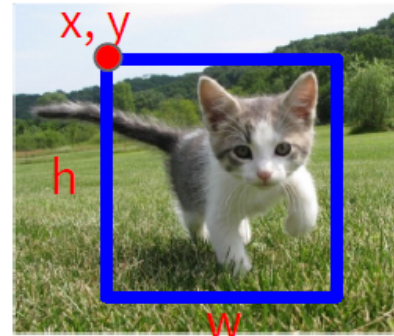
Performance Evaluation of Localization



images with multiple objects: number of detections dependent on used confidence threshold

Object Detection: Single Object

(Classification + Localization)



Fully
Connected:
4096 to 1000

Class Scores

Cat: 0.9
Dog: 0.05
Car: 0.01
...

Correct label:
Cat

Softmax
Loss

Multitask Loss

+ → Loss

Vector:
4096

Fully
Connected:
4096 to 4

Box
Coordinates
(x, y, w, h)

L2 Loss

Correct box:
(x', y', w', h')

Treat localization as a
regression problem!

too complicated for multiple objects

Localization for Multiple-Object Images

idea: classify many different crops of the image as object or background
(crops: sliding window over image, iterated at multiple window sizes)



Dog? No
Cat? No
Background? Yes



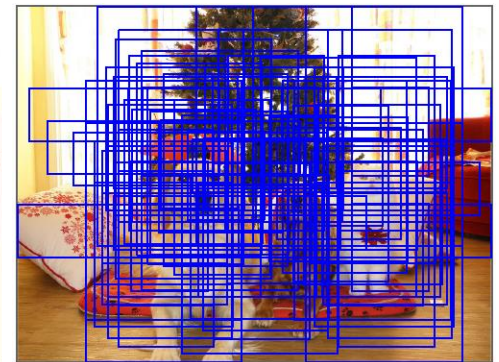
Dog? Yes
Cat? No
Background? No



Dog? Yes
Cat? No
Background? No



Dog? No
Cat? Yes
Background? No



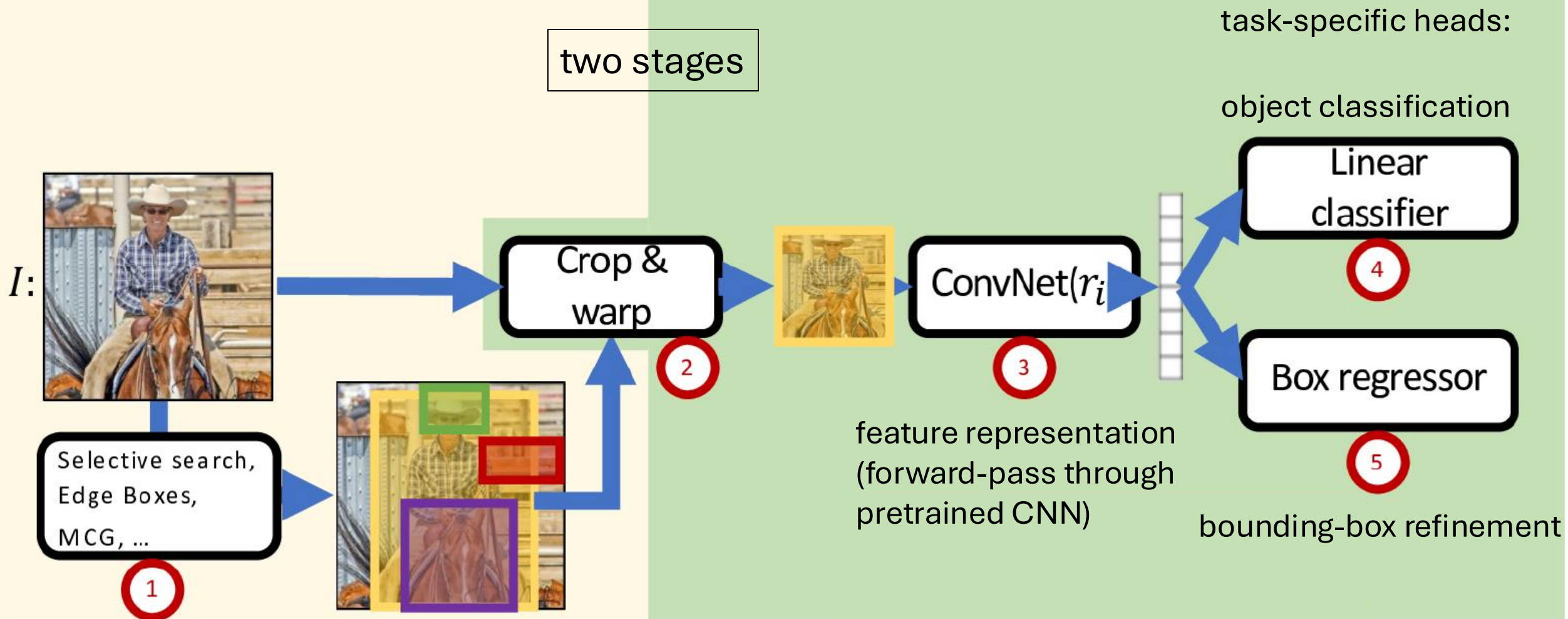
issue: too many
possible crops

→ need for region proposals

Region-Based Convolutional Neural Network (R-CNN)

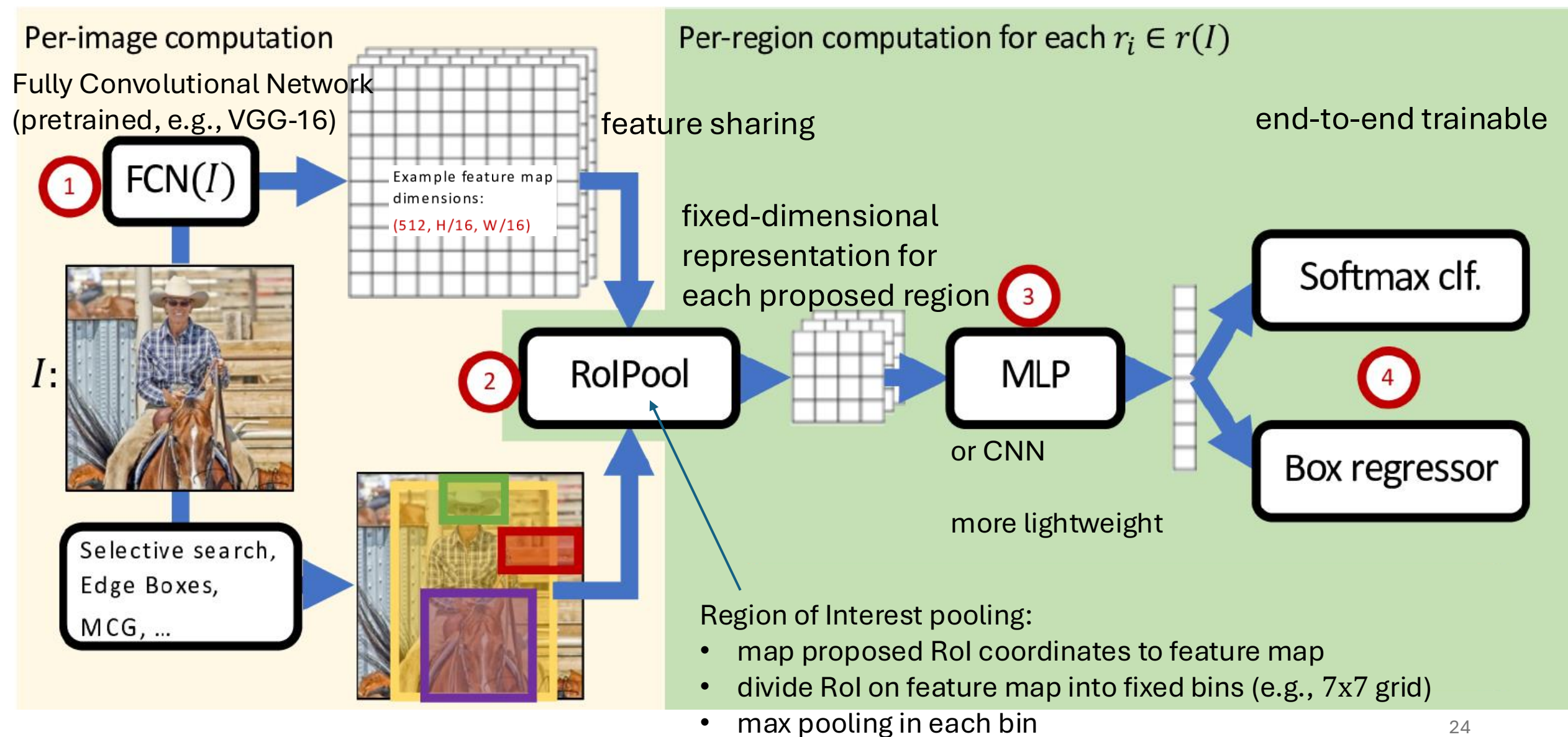
Per-image computation

Per-region computation for each $r_i \in r(I)$



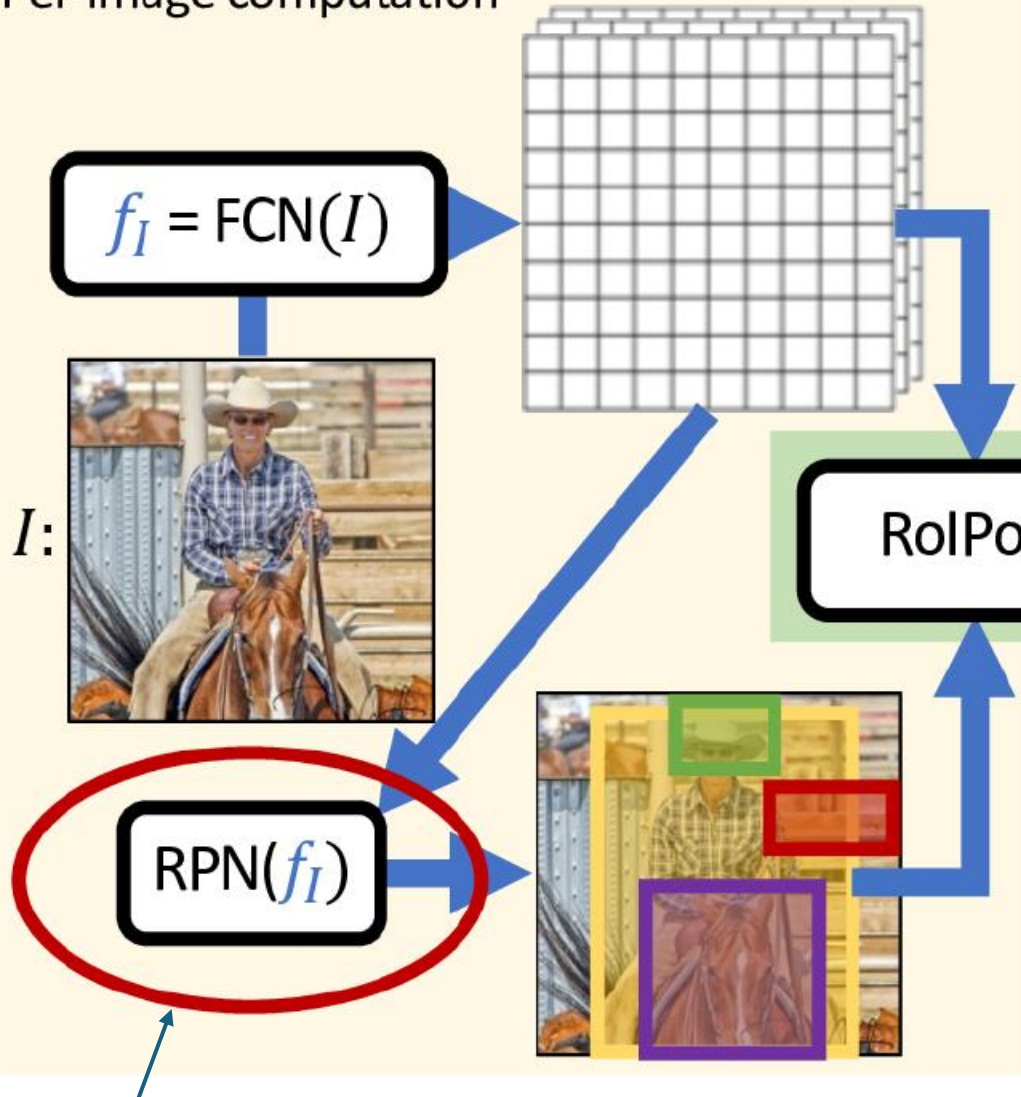
region proposals by "classic" method ($\sim 2k$)

Fast R-CNN: Crop ConvNet Features



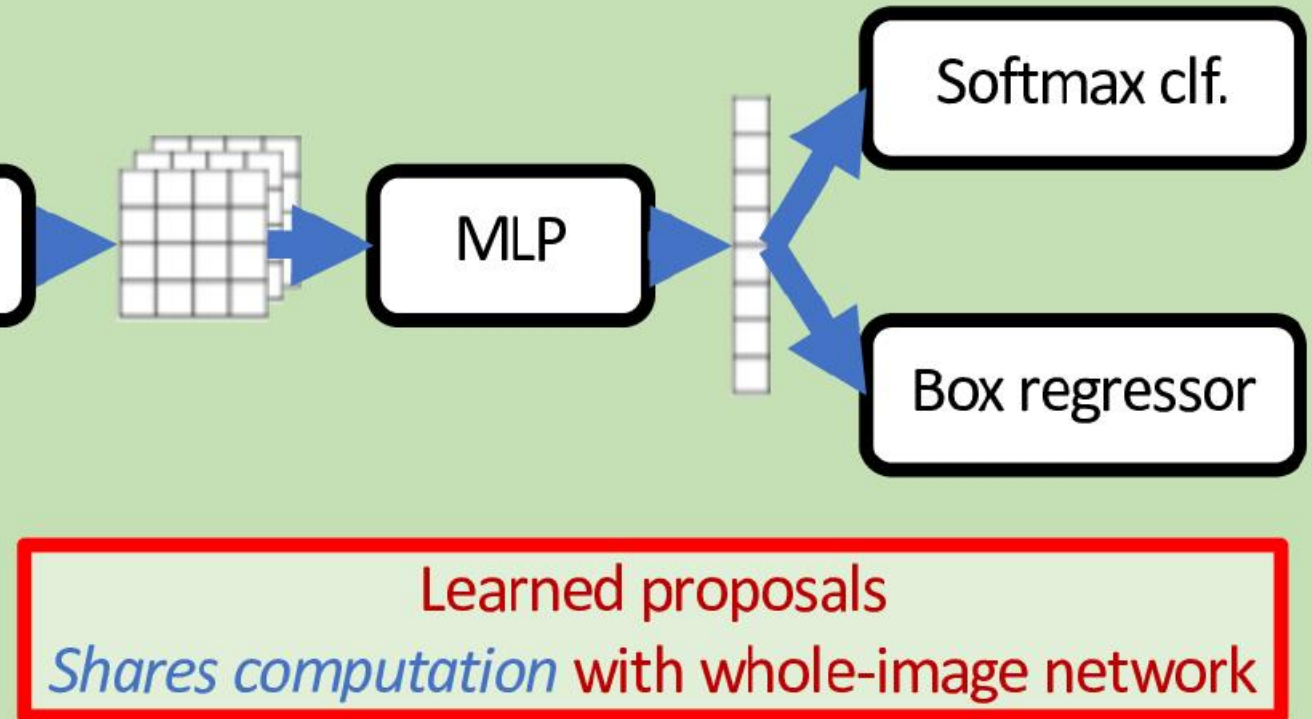
Faster R-CNN: Use Region Proposal Network

Per-image computation



Per-region computation for each $r_i \in r(I)$

two RPN losses combined with two detection head losses during end-to-end training

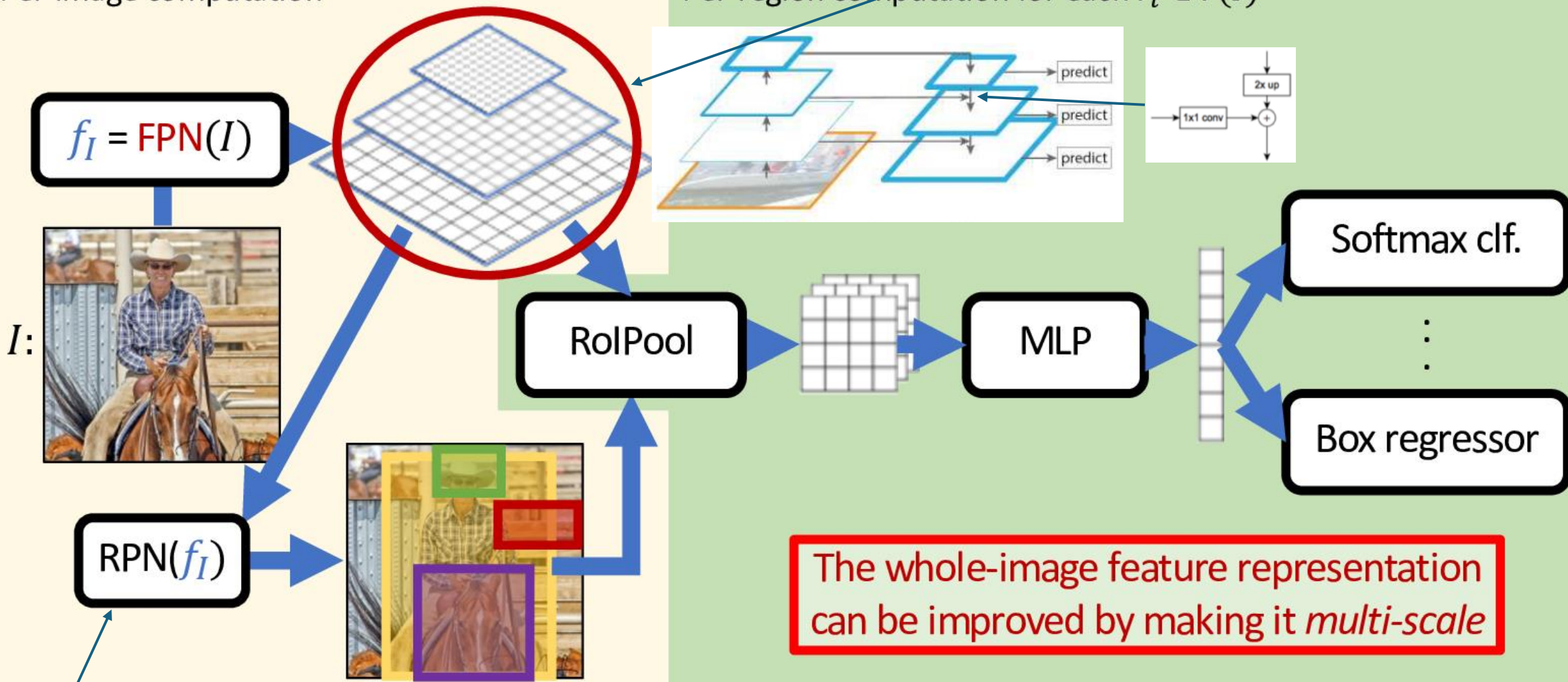


Feature Pyramid Network (FPN)

different intermediate layers
of same backbone CNN

Per-image computation

Per-region computation for each $r_i \in r(I)$

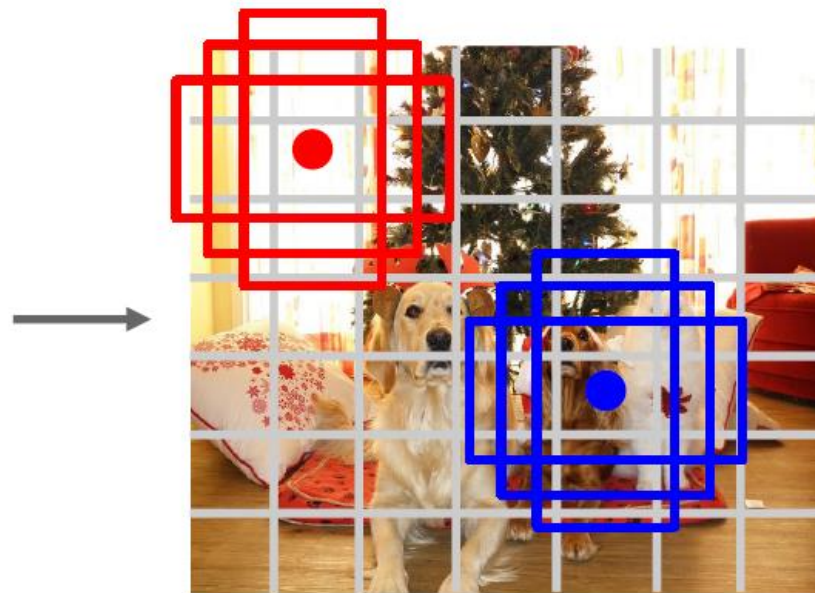


using different anchor sizes for different pyramid levels

Single-Stage Detectors: Drop Per-Region Computation



Input image
 $3 \times H \times W$



Divide image into grid
 7×7

Image a set of base boxes
centered at each grid cell
Here $B = 3$

Within each grid cell:

- Regress from each of the B base boxes to a final box with 5 numbers:
($dx, dy, dh, dw, confidence$)
- Predict scores for each of C classes (including background as a class)
- Looks a lot like RPN, but category-specific!

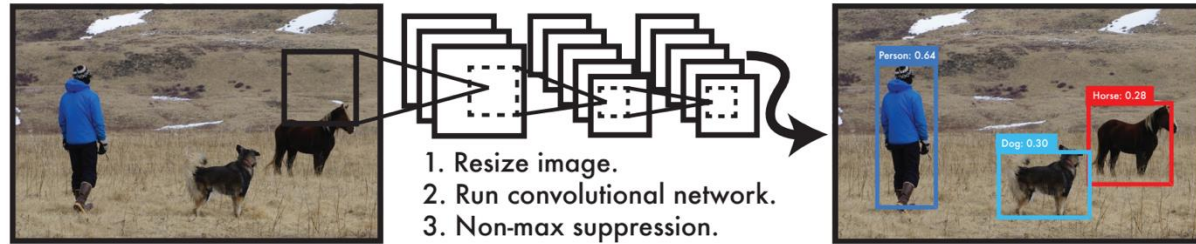
Output:
 $7 \times 7 \times (5 * B + C)$

Redmon et al, "You Only Look Once:
Unified, Real-Time Object Detection", CVPR 2016
Liu et al, "SSD: Single-Shot MultiBox Detector", ECCV 2016
Lin et al, "Focal Loss for Dense Object Detection", ICCV 2017

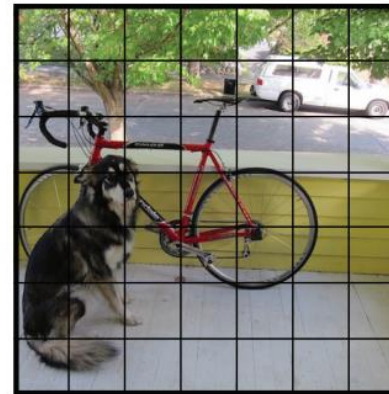
faster, but usually worse performance

YOLO: Real-Time Object Detection

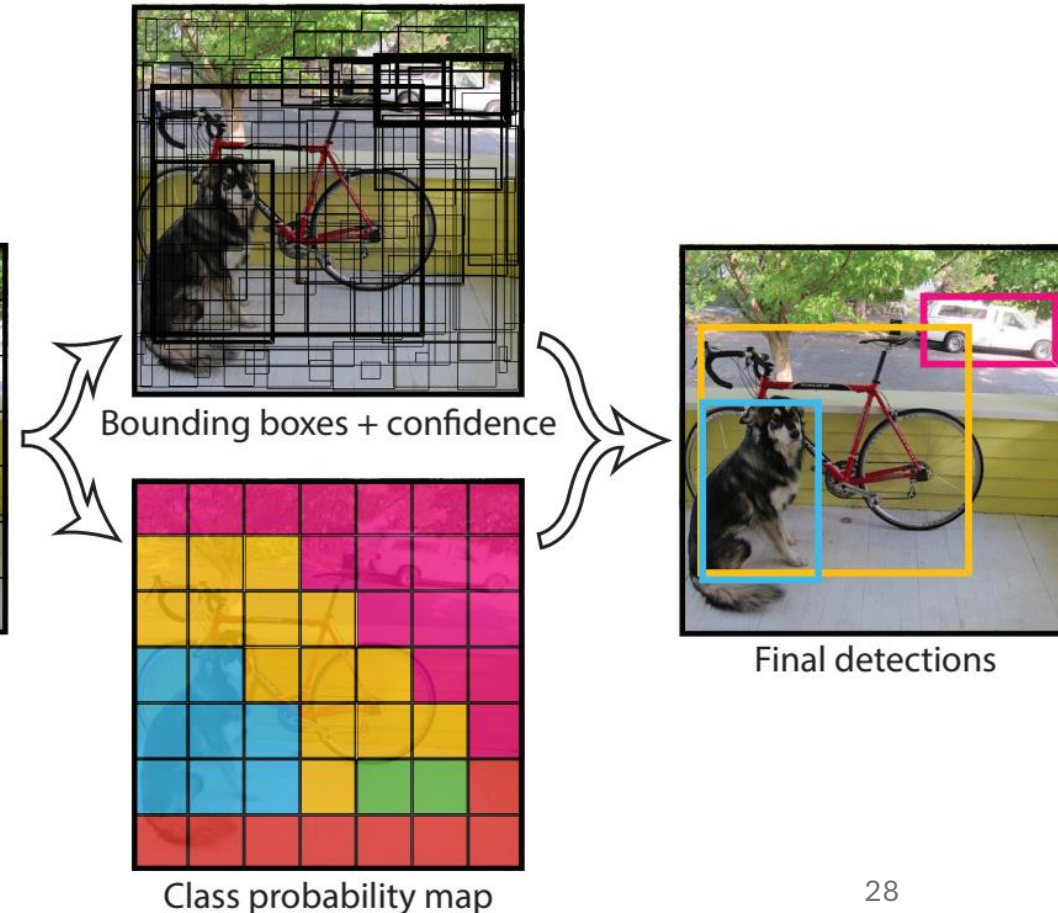
anchor-free approach
in newer YOLO versions



You Only Look Once:
prediction of bounding boxes
and class probabilities in one go



$S \times S$ grid on input



Object Tracking

task: locating an object in successive video frames

basic approach:

1. detection: localize target object(s) (e.g., YOLO)
2. motion estimation: predict next location (e.g., Kalman filter or optical flow)
3. appearance matching: comparison of previous and predicted next location (e.g., feature descriptor, CNN features)

detection repeated frequently to correct for accumulated deviations

Optical Flow

dense optical flow:

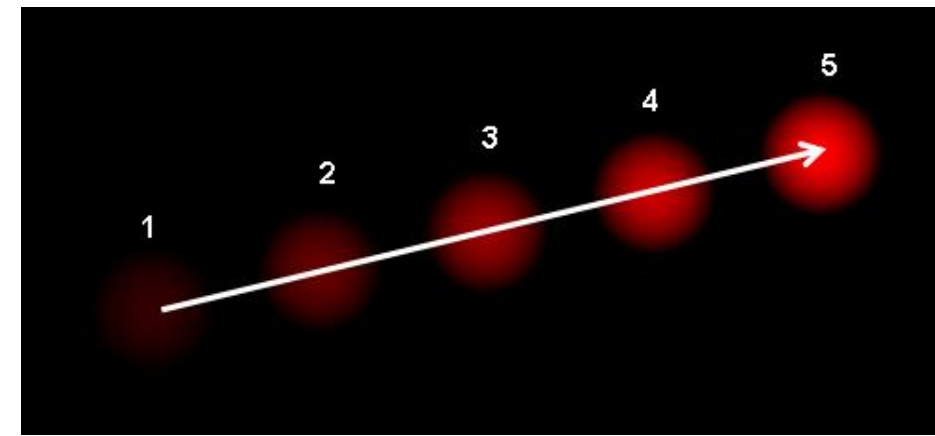
estimate motion vector of every pixel

e.g., Horn-Schunck method or Lucas-Kanade (LK) method

sparse optical flow:

estimate motion vector of few image features

e.g., Kanade-Lucas-Tomasi (KLT) feature tracker



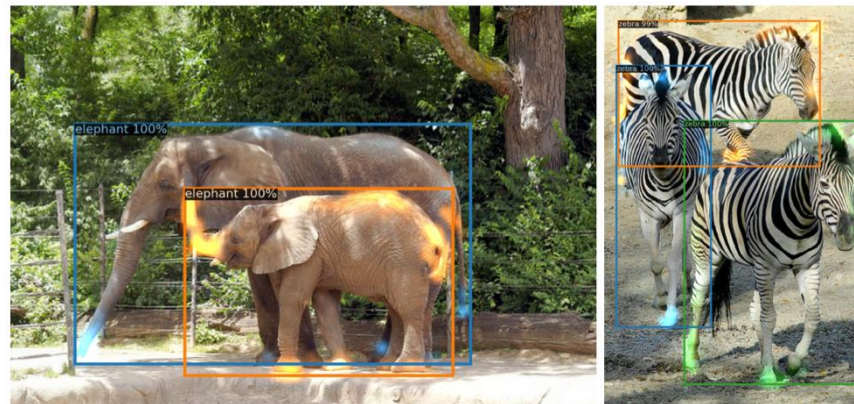
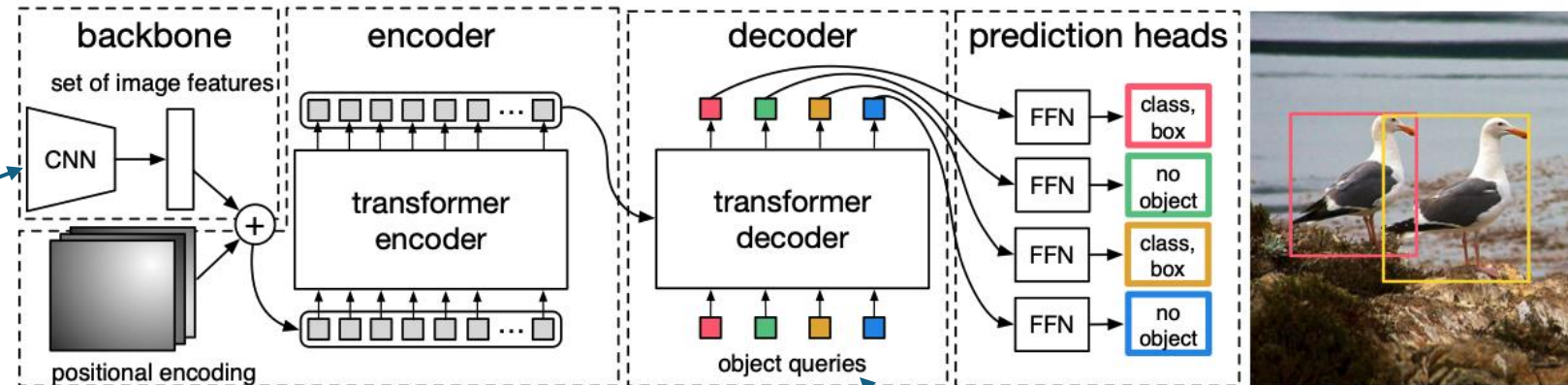
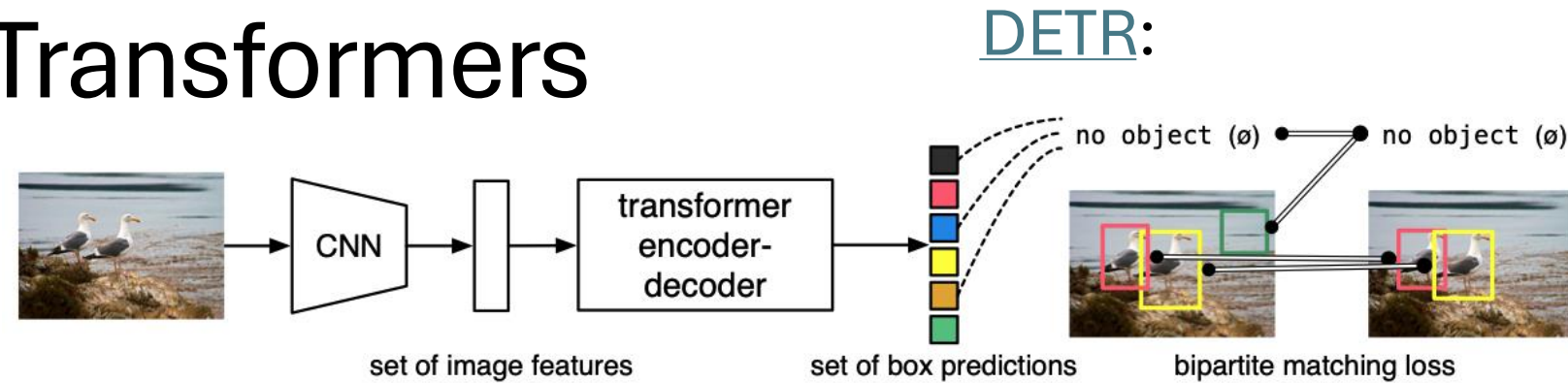
Detection with Transformers

idea: use a transformer to directly predict object bounding boxes and class labels from image features (no RPN)

backbone can also be vision transformer

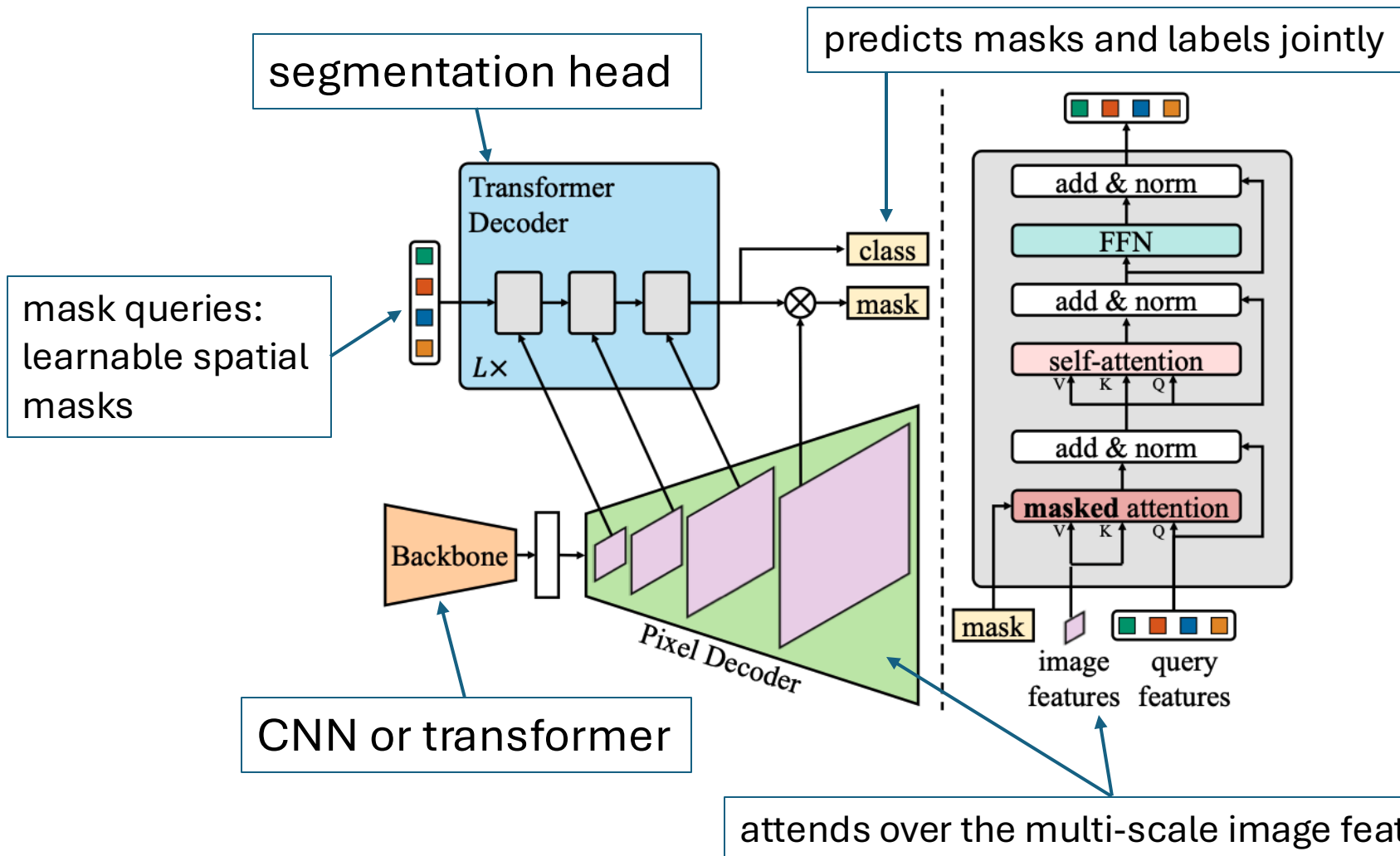
→ popular combination: DINOv2 + DETR

visualized decoder attention for predicted objects:



slot for learnable vectors
("Is there an object here?")

Segmentation with Transformers



Mask2Former

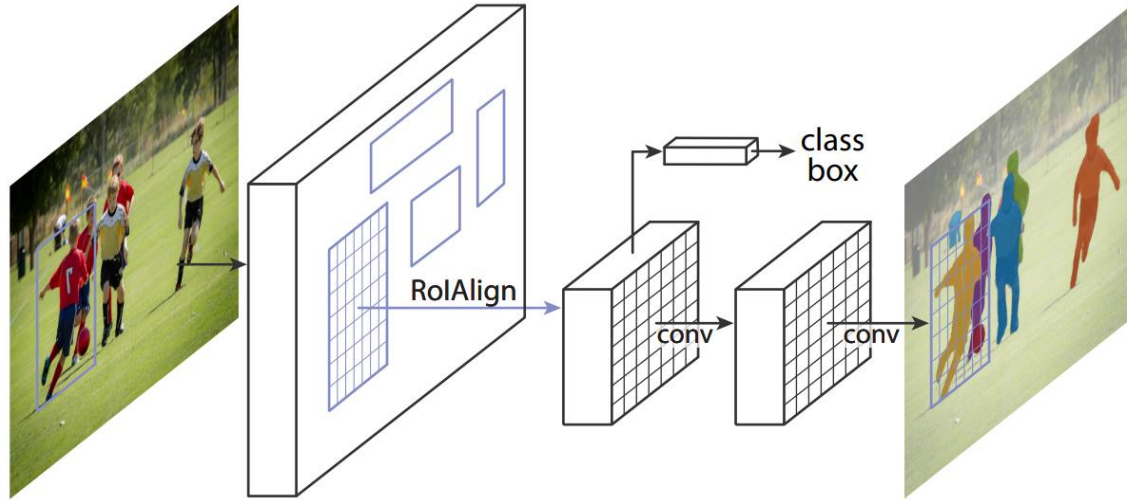
unlike pixel-wise classification, it predicts a set of masks, each tied to an object or region

(similar to DETR, but predicting segmentation masks instead of bounding boxes)

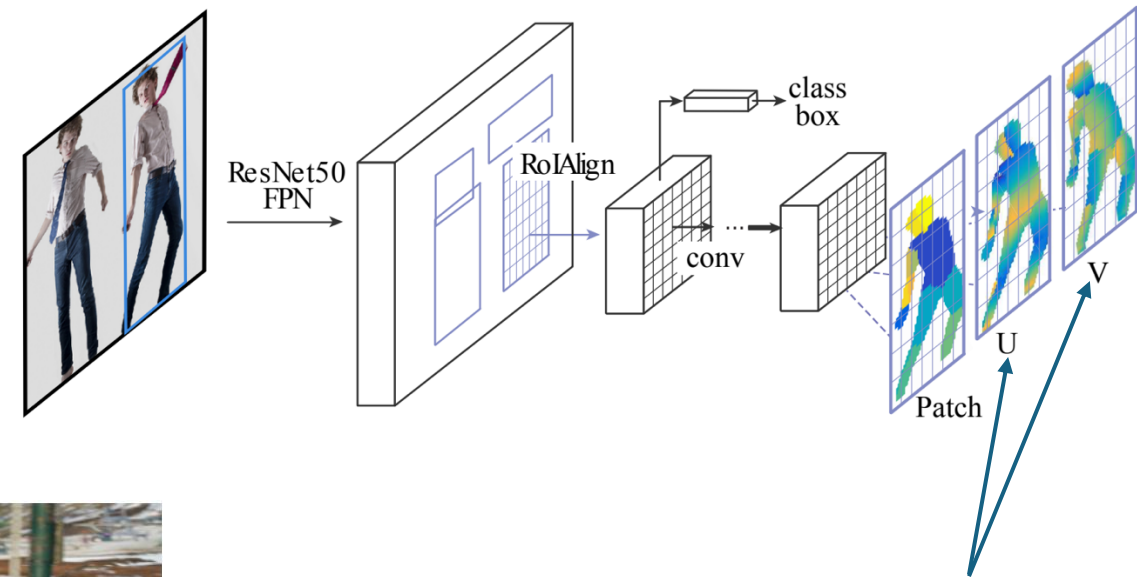
Instance Segmentation

Add Additional Network Heads to R-CNN

instance segmentation ([Mask R-CNN](#)):



pose estimation ([DensePose](#)):

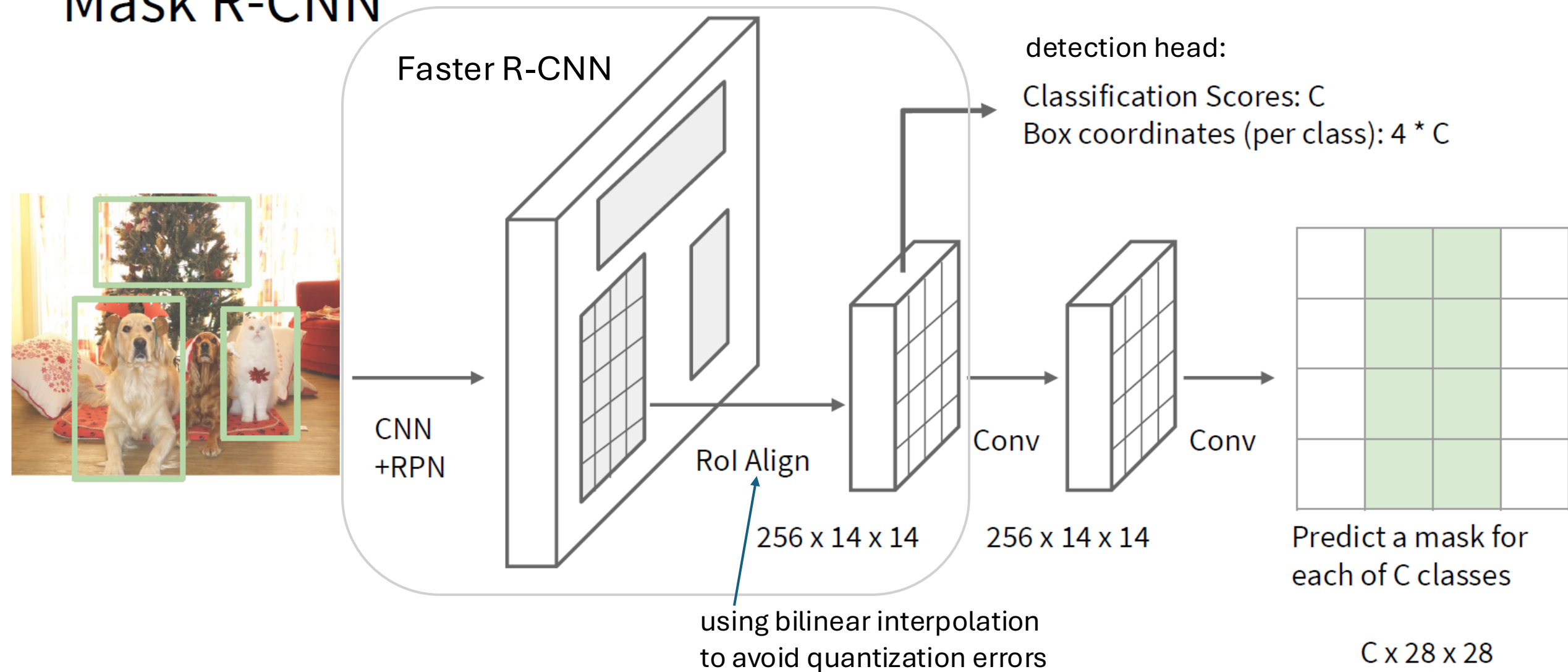


3D model's surface to 2D



Mask R-CNN

→ total loss is sum of RPN, detection-head, and mask-head losses



1. object detection (boundary boxes)
2. prediction of separate binary masks for each detected object (specific instances)³⁵

image with training proposal

28 × 28 mask target

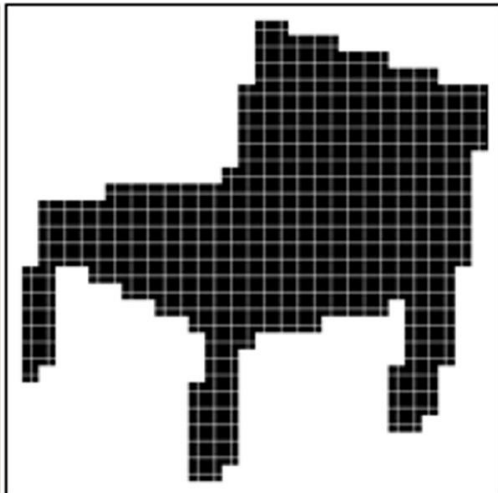
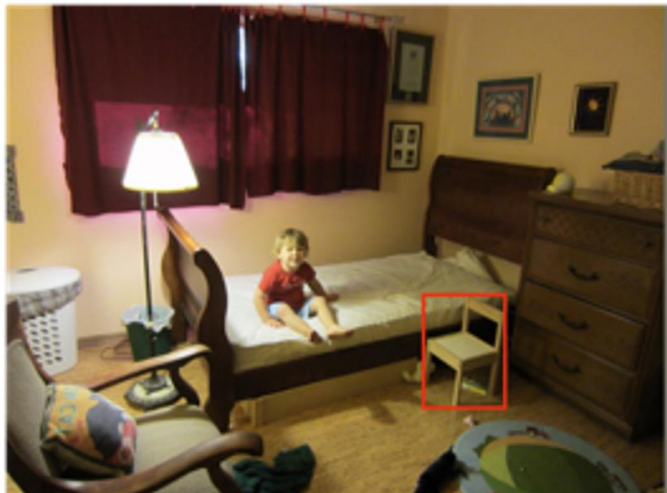
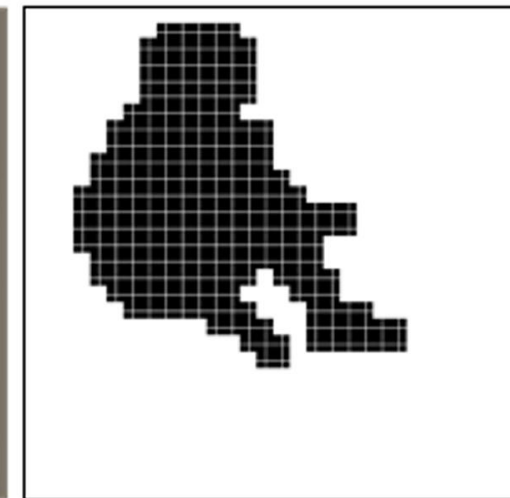
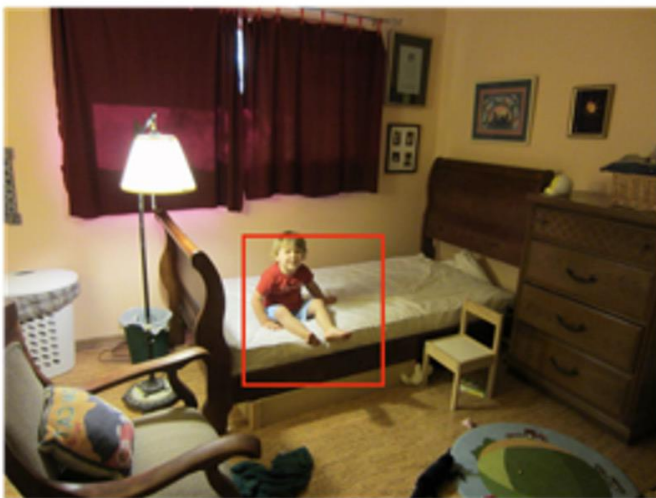
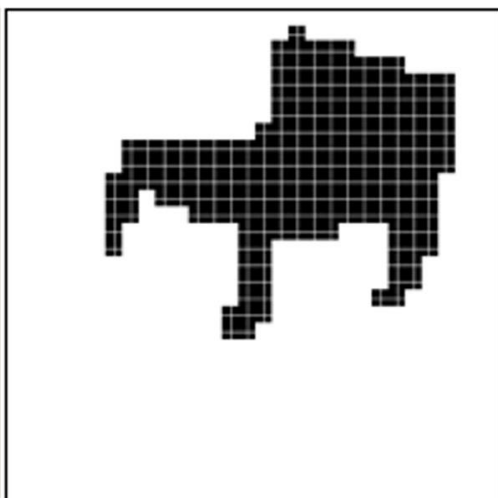
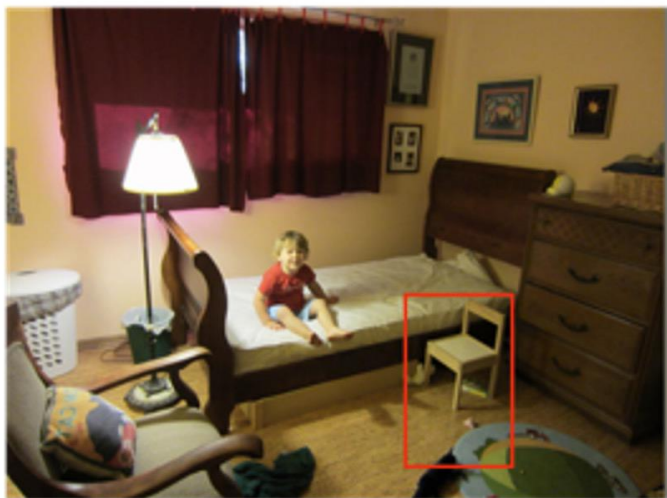
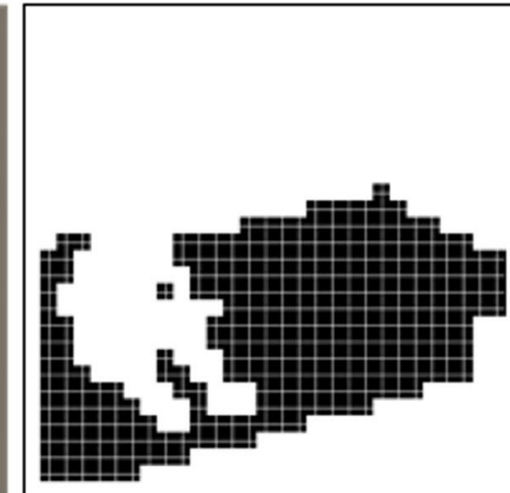
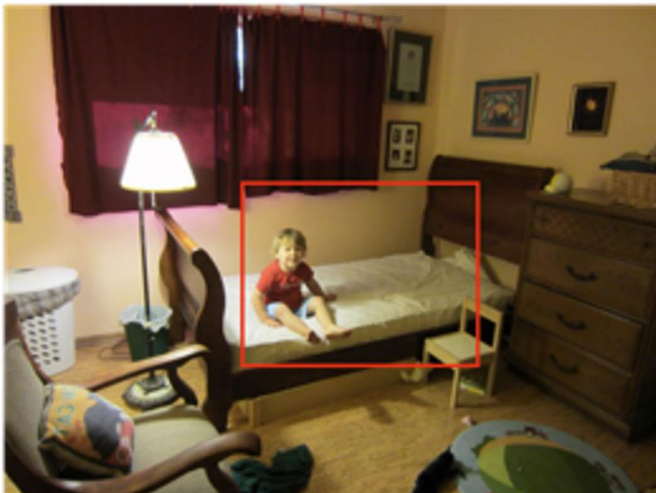
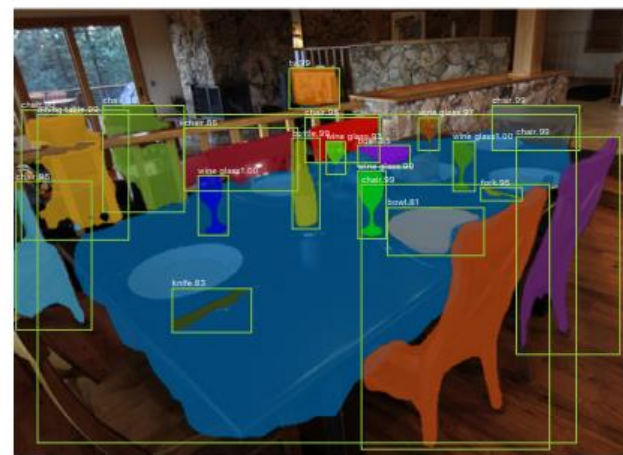
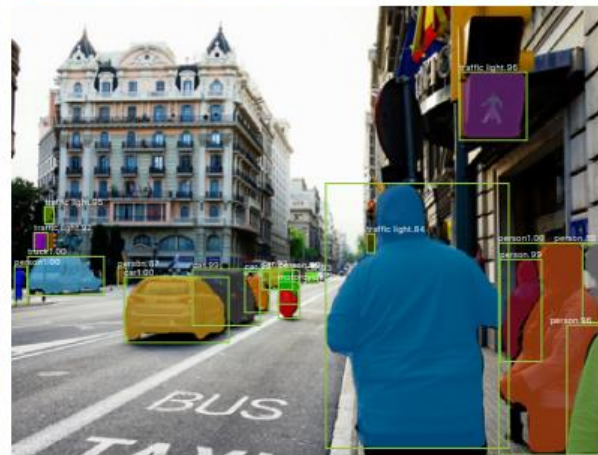
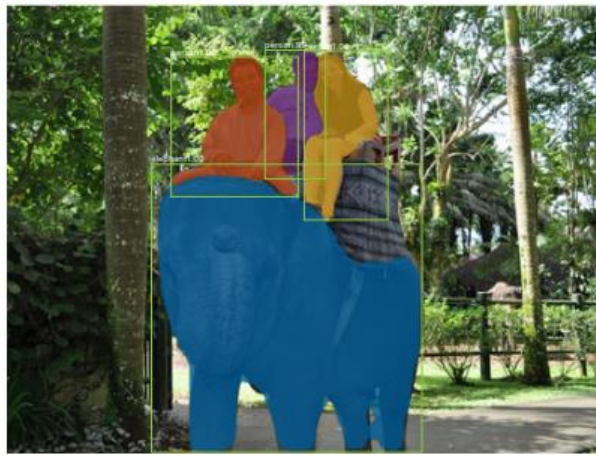
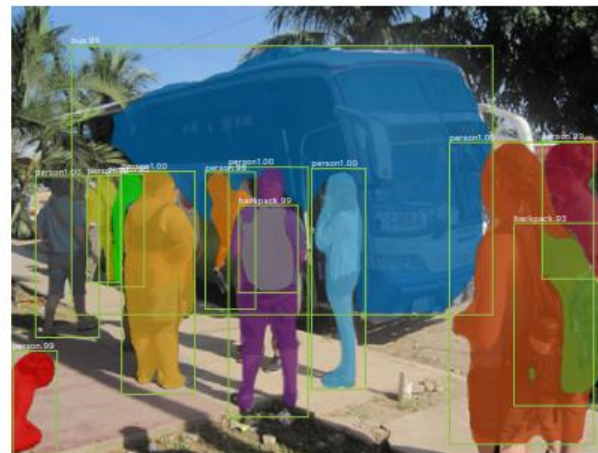
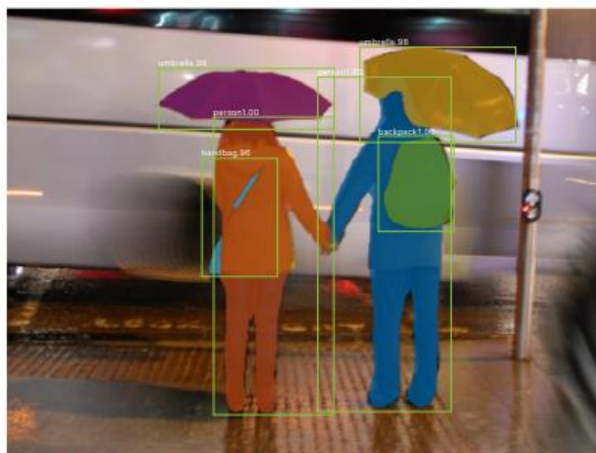
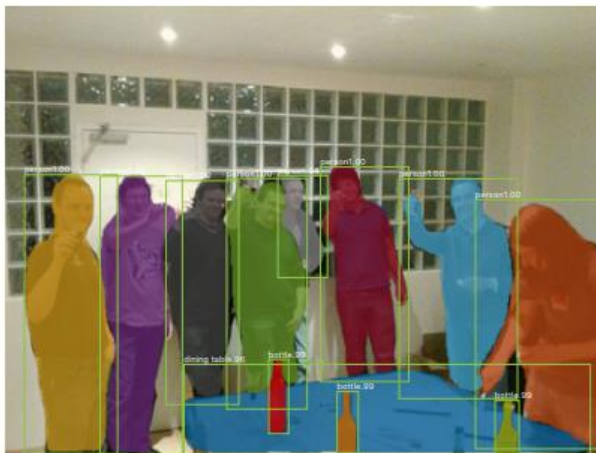


image with training proposal

28 × 28 mask target



results on MS COCO data set



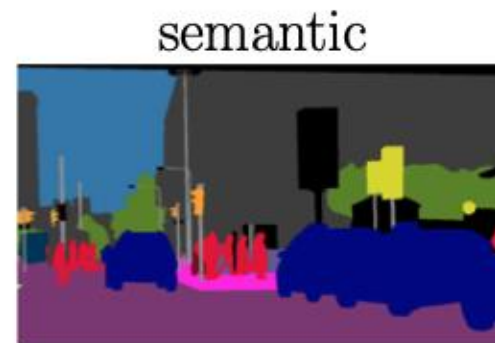
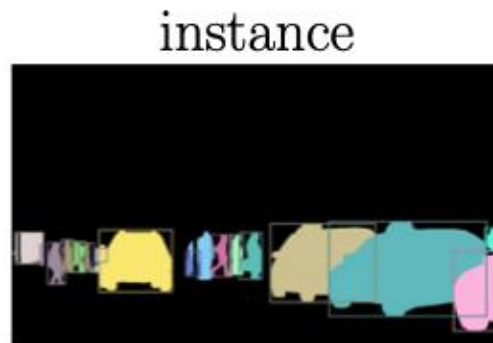
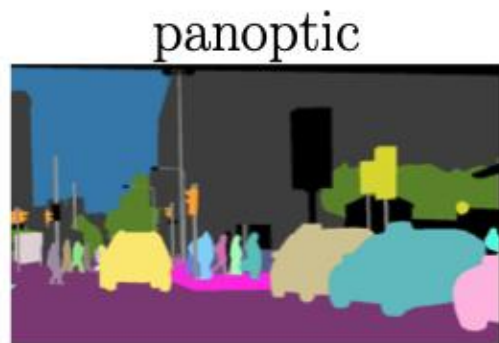
Panoptic Segmentation

unification of semantic segmentation (labeling all pixels) and instance segmentation (identifying individual objects)

each pixel is assigned

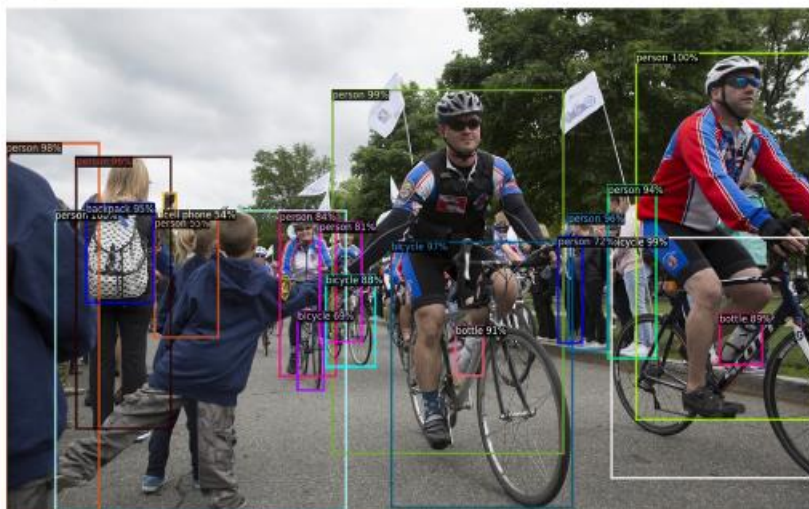
- a class label (e.g., road, car)
- an instance ID, if it belongs to a “thing” class (e.g., car #1, car #2)

covering both “thing” classes (countable objects) and “stuff” classes (uncountable regions like sky, grass, road)



e.g., with combination of Mask R-CNN and fully-convolutional head (or DETR variants such as Mask2Former)

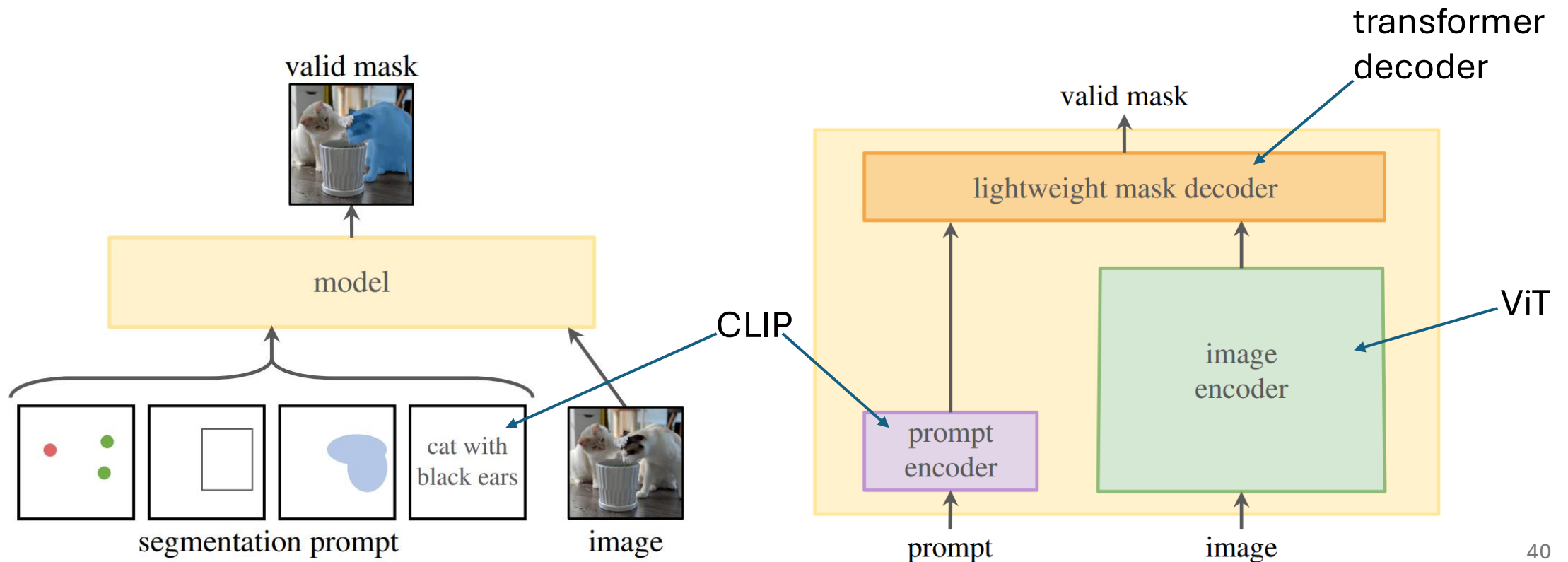
Detectron2: PyTorch Implementations



Promptable Segmentation with Transformers

Segment Anything Model ([SAM](#))

[SAM2](#) includes also video segmentation



Some Off-the-Shelf ML Methods

multivariate tabular data prediction: HistGradientBoosting from scikit-learn ([regression](#), [classification](#))

image classification: feature extraction from [DINOv2](#) or finetune a ResNet/ViT from [torchvision](#) (zero-shot: [CLIP](#))

object detection: Faster R-CNN ResNet-50 FPN from torchvision (zero-shot: [YOLOv8](#))

semantic segmentation: FCN ResNet-50 from torchvision (zero-shot: YOLOv8-seg)

instance segmentation: Mask R-CNN ResNet-50 FPN from torchvision (zero-shot: YOLOv8-seg or [SAM](#))

text classification: finetune [DistilBERT](#) from transformers (zero-shot: prompt an [Ollama](#) LLM)